

RTL Design Sherpa

Binary BCH UART Harness Genesys 2 FPGA board evidence

October 2026

Table of Contents

1 Document Information....7
1.1 Revision History....7
1.2 Scope....8
1.3 Provenance....9
2 Purpose and Geometry....10
3 What Is In It....11
3.1 RTL blocks....11
3.2 Host scripts....13
3.3 Make targets and build flow....14
4 How The Pieces Connect....15
4.1 Register path....15
4.1.1 Figure 2.1: BCH AXIS datapath and register path....16
4.1.2 Figure 2.3: UART harness control path....17
4.2 AXI4 flavour datapath....18
4.2.1 Figure 2.2: BCH AXI4 memory-to-memory datapath....19
4.3 Clock and reset....21
5 Address Map....22
5.1 Figure 2.4: UART harness address map....23
5.2 Window summary....24
5.3 Loop CSR registers (window 0)....25
5.3.1 Identification and control....25
5.3.2 Status and profile....26

5.3.3 Bandwidth meters....26
5.4 Observer windows (windows 1 and 2)....28
6 The FPGA-Testing Choices, And Why....29
6.1 Deterministic campaigns....29
6.2 Injector modes mapped to real failure mechanisms....30
6.3 Post-encoder injection protecting parity bits too....31
6.4 Recheck + flip-only verdict as the on-chip oracle....32
6.5 UART over a debug core, plus the safety pair....33
6.6 AXIS vs AXI4 flavours covering both integration styles....34
7 Operating It....35
8 What Is Deliberately Not Board-Tested....36

List of Figures

Figure 2.1: BCH AXIS datapath and register path.....16
Figure 2.3: UART harness control path.....17
Figure 2.2: BCH AXI4 memory-to-memory datapath.....19
Figure 2.4: UART harness address map.....23

List of Tables

Table 1: Table 0.1: Revision history.....7

1 Document Information

1.1 Revision History

Table 0.1: Revision history.

Revision	Date	Author	Description
0.1	2026-10-06	RTL Design Sherpa	Initial harness guide for the BCH Genesys 2 UART loop.
0.2	2026-10-20	RTL Design Sherpa	Block diagrams redrawn to the repo element key (v3): class fills, double-bordered registered blocks, green return paths and a generated key under each figure. No technical content changed.

1.2 Scope

This book describes the BCH(4224,4120) t=8 UART harness on the Digilent Genesys 2: its contents, connectivity, and the motivations behind the FPGA testing choices. It is the harness guide that pairs with the Binary BCH Board Validation Report, which carries the run evidence.

1.3 Provenance

The harness description comes from `docs/UART_HARNESS.md`, the living markdown copy in the harness directory. Build facts come from `stable/MANIFEST.md` and `stable/reports/`. The board used for the runs is Digilent Genesys 2 serial `200300B818A0` (Kintex-7 `xc7k325t-2`), programmed and measured with Vivado 2025.1.

2 Purpose and Geometry

This is the BCH(4224,4120) t=8 codec on the Digilent Genesys 2, wrapped in a UART-controlled loop that runs the same campaigns in cocotb simulation and on the board. The geometry is frozen in `build-loop/rtl/bch_loop_cfg_pkg.sv`: GF(2^13), primitive polynomial `0x201B`, first root `b=1`, `n=4224` shortened from `8191 = 2^13 - 1`, parity degree `104 = m*t`. The component core defaults to `BITS_PER_BEAT = 8`; this harness widens the AXI-Stream/AXI4 wrappers to 32-bit beats.

The whole point of the loop is that errors are injected **after** the encoder. The checker regenerates the expected LFSR pattern independently and compares it against the decoder's output, so the verdict comes from a reference that never saw the errors.

The harness directory moved from `NexysA7/` to `Genesys2/` on 2026-10-05. `BCH_TARGET` still accepts `nexys_a7_100t` as a build option, and since 2026-10-07 the same tree carries a first-class small profile for it (issue #82): `BCH_PROFILE=small` builds BCH(248,224) t=3 for the A7-100T at 50 MHz (WNS +7.8 ns, 38% LUT). The Genesys 2 remains the primary target for the full profile: the k325t-2 has the room and timing margin to hold the full BCH(4224,4120) loop at 100 MHz with both AXIS and AXI4 flavours, and both images came out timing-clean on the board build with positive slack.

Two bitstreams are built: `bch_loop_genesys2_axis.bit` and `bch_loop_genesys2_axi4.bit`. The AXIS flavour is a single streaming pipe; the AXI4 flavour is a memory-to-memory job chain across four AXI4 memories. The AXI4 flavour refuses runs whose `GEN_BLOCKS` exceed the job-memory capacity rather than letting regions wrap, so long soaks belong on the AXIS image. The host reads `TOPOLOGY` to discover which datapath is loaded, so the same campaign scripts drive both without a command-line flag.

3 What Is In It

3.1 RTL blocks

Module	File	Role
`bch_loop_genesys2_top`	`build-loop/rtl/bch_loop_genesys2_top.sv`	Genesys 2 board top: 200 MHz LVDS input, MMCM to 100 MHz, UART, LEDs.
`bch_loop_top`	`build-loop/rtl/bch_loop_top.sv`	Nexys A7-100T board top, kept as the `nexys_a7_100t` target option.
`bch_loop_harness`	`build-loop/rtl/bch_loop_harness.sv`	Board-agnostic datapath, CSR wiring, observer mux, status aggregation.
`bch_axi4_pipeline`	`build-loop/rtl/bch_axi4_pipeline.sv`	AXI4 flavour: memory-to-memory codec chain over three job memories.
`bch_loop_cfg_pkg`	`build-loop/rtl/bch_loop_cfg_pkg.sv`	One source of geometry, UART baud, build ID `0x4243_4850` (“BCHP”).
`bch_loop_regs` / `bch_loop_regs_pkg`	`build-loop/rtl/generated/bch_loop_regs/rtl/bch_loop_regs.sv`, `bch_loop_regs_pkg.sv`	PeakRDL-generated CSR block from `build-loop/rtl/bch_loop_regs.rdl`.
`bridge_bch_loop_axil`	`rtl/bridges/generated/bridge_bch_loop_axil/bridge_bch_loop_axil.sv`	Generated 1-master x 3-slave AXIL-to-APB fabric.
`uart_axil_bridge`	`projects/components/utility-ip/converters/rtl/uart_to_axil4/uart_axil_bridge.sv`	UART byte stream to AXI4-Lite master.
`apb4_to_peakrdl`	`projects/components/utility-ip/converters/rtl/apb4_to_peakrdl.sv`	APB4 slave to PeakRDL cpuif shim.

Module	File	Role
axis4_master_injector	rtl/amba/shared/axis4_master_injector.sv	LFSR data source, one packet per block, per-channel CRC-32.
bch_encoder_axis4 / bch_encoder_core	projects/components/ecc-ip/bch/rtl/top/bch_encoder_axis4.sv, rtl/macro/bch_encoder_core.sv	BCH encoder, AXIS wrapper around the core.
bch_beat_packer	projects/components/ecc-ip/bch/rtl/fub/bch_beat_packer.sv	Repacks encoder output into ceil(n/BITS_PER_BEAT) codeword beats.
error_injector	projects/components/utility-ip/misc/rtl/error_injector.sv	Shared bit-granular injector, modes 0..7, sits after the encoder.
bch_decoder_axis4 / bch_decoder_core	projects/components/ecc-ip/bch/rtl/top/bch_decoder_axis4.sv, rtl/macro/bch_decoder_core.sv	riBM decoder, AXIS wrapper around the core.
axis4_slave_pattern_check	rtl/amba/shared/axis4_slave_pattern_check.sv	Independent reference checker, byte-granular CRC-32.
axis4_intf_observer	projects/components/utility-ip/misc/rtl/axis4_intf_observer.sv	APB-programmable observer on the four AXIS seams.
axi4_intf_master_observer	projects/components/utility-ip/misc/rtl/axi4_intf_master_observer.sv	APB-programmable observer on the AXI4 pipeline ports (AXI4 flavour only).
axi_bus_meter	rtl/amba/shared/axi_bus_meter.sv	Four windowed bandwidth meters inside the harness.

3.2 Host scripts

Script	File	Role
bch_env.py	bin/bch_env.py	Path anchor: puts the shared projects/fpga-systems/bin and build-loop/host on sys.path.
run_smoke.py	bin/run_smoke.py	Campaign runner: resolves sequences, opens BchLoopDriver, prints report.
Init	bin/seq_init.py	Proves BUILD_ID, SCRATCH round-trip, PROFILE, TOPOLOGY.
Smoke	bin/seq_smoke.py	Bypass, clean, e=t, e=t+1, DEBUG walk.
Sweep	bin/seq_sweep.py	Exact count for e = 0 .. 2t+2.
RandomCampaign	bin/seq_random.py	Fresh data/error seeds, mixed modes.
Soak	bin/seq_soak.py	Long-running random-pattern stress.
BchLoopDriver / RunResult	build-loop/host/bch_loop.py	By-name register access, status collection, observer readers.
bch_loop_programs	build-loop/host/bch_loop_programs.py	The authored-once programs: smoke, bypass, run, sweep, verdict.
host_bch_loop.py	build-loop/host/host_bch_loop.py	CLI front-end over the same programs.
bch_loop_regs_regmap.py	build-loop/dv/tbclasses/bch_loop_regs_regmap.py	PeakRDL-generated by-name register map.

3.3 Make targets and build flow

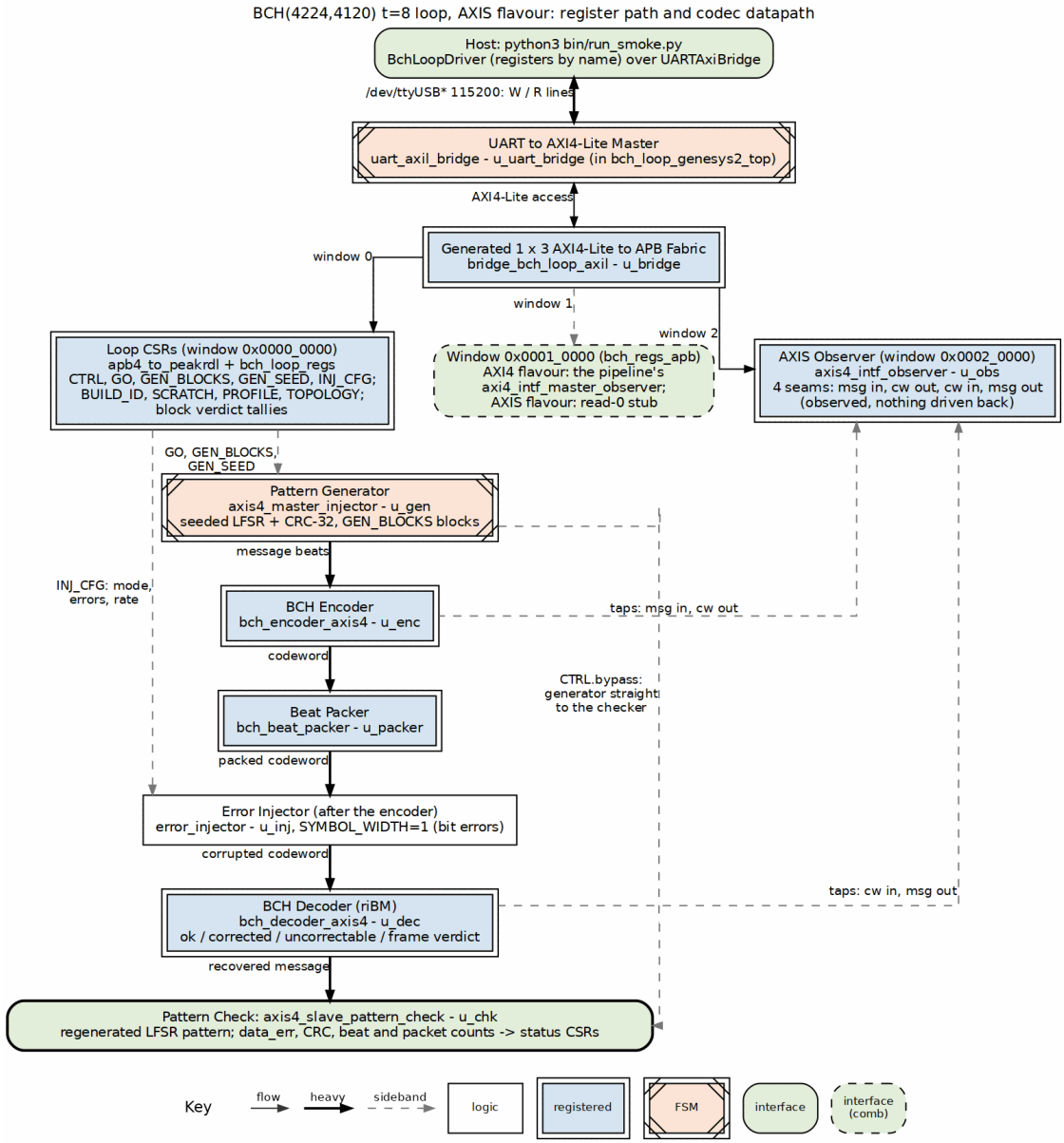
Make target	File	Role
`make help`	`Makefile`	Area dispatcher; delegates every target to `build-loop/`.
`make -C build-loop bitstream`	`build-loop/Makefile`	Vivado build; set `BCH_TARGET=genesys2` and `BCH_IFACE={AXIS,AXI4}`.
`make -C build-loop program`	`build-loop/Makefile` → `make/fpga_board.mk` → `projects/fpga-systems/bin/fpga_board.py`	Programs the board after JTAG identity verification.
`make regmap`	`Makefile`	Regenerates the CSR RTL and `dv/tbclasses/bch_loop_regs_regmap.py` from the RDL.
`bin/regen_bridges.sh`	`bin/regen_bridges.sh`	Regenerates the 1x3 fabric from `rtl/bridges/configs/bridge_bch_loop_axil.toml`.
`bin/build_image_matrix.sh`	`bin/build_image_matrix.sh`	Serial build of both `bch_loop_genesys2_axis.bit` and `bch_loop_genesys2_axi4.bit`.

4 How The Pieces Connect

4.1 Register path

The host talks by name, not by offset: `BchLoopDriver` wraps `UARTAxiBridge` with `UartRegisterMap` over `bch_loop_regs_regmap.py`. The diagram shows the path from the laptop to the three APB windows.

4.1.1 Figure 2.1: BCH AXIS datapath and register path

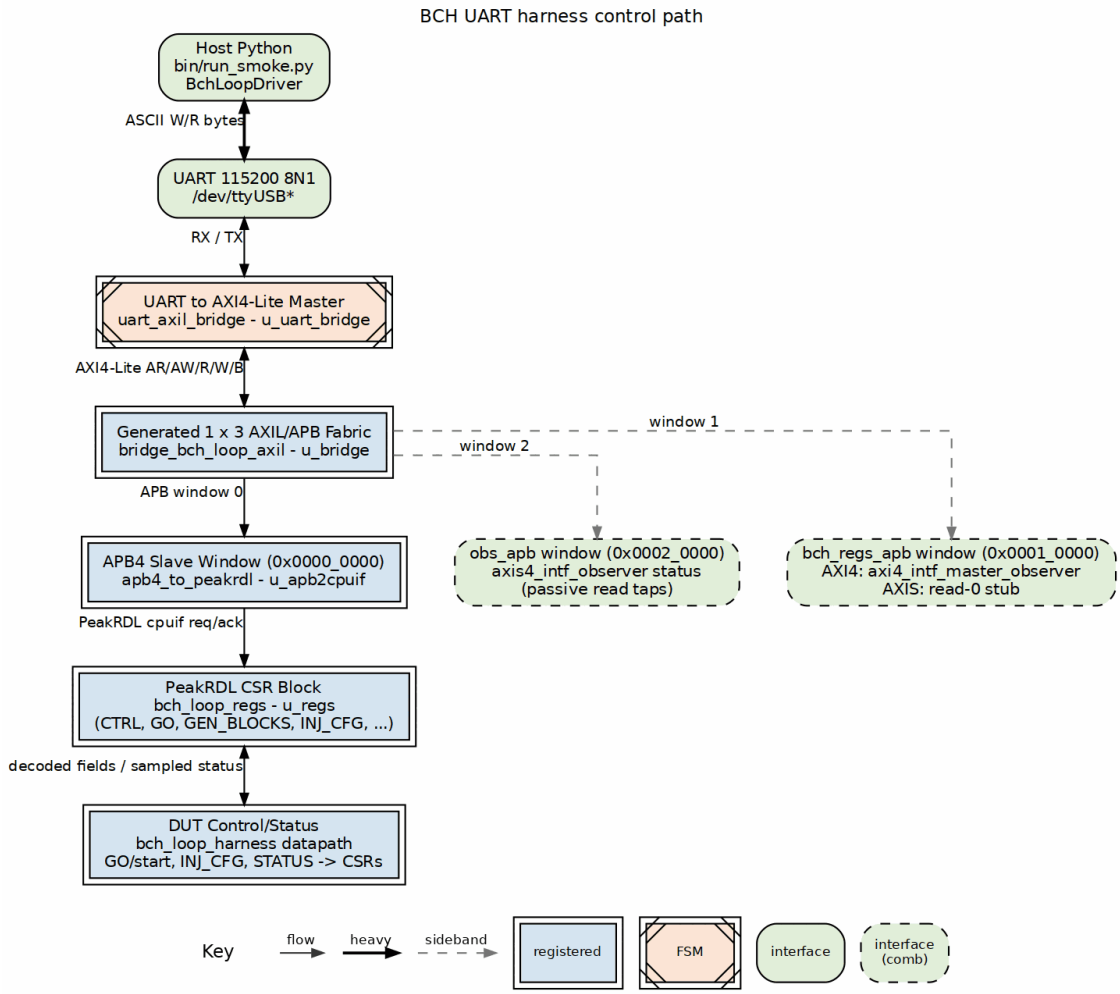


BCH AXIS datapath and register path

Source: 01_bch_axis_loop.dot

The register path is the same in both flavours. The only raw addresses in the host code are the two expansion-window bases used to attach observer register maps: `OBS_AXI4_BASE = 0x00010000` and `OBS_AXIS_BASE = 0x00020000` in `build-loop/host/bch_loop.py`. Those bases match `rtl/bridges/generated/bridge_bch_loop_axil/bridge_bch_loop_axil.toml`; if a window moves, the bridge is regenerated and the host picks it up in one place.

4.1.2 Figure 2.3: UART harness control path



UART harness control path

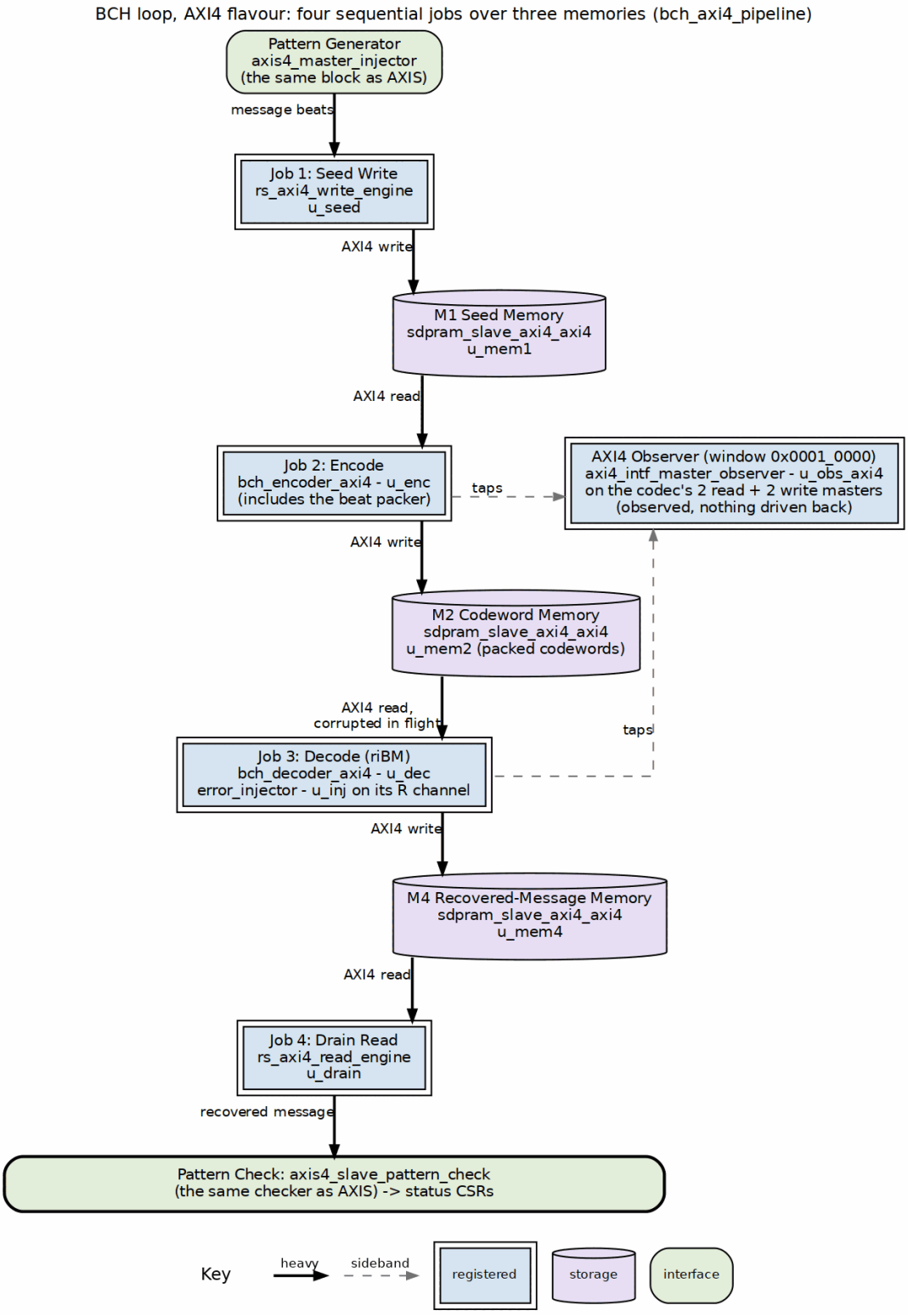
Source: 03_uart_harness_control.dot

This diagram isolates the command/status path: host Python → UART → `uart_axil_bridge` → `bridge_bch_loop_axil` → APB window → `apb4_to_peakrdl` → `bch_loop_regs` → harness control and status. The datapath blocks drawn in Figure 2.1 and Figure 2.2 receive their control from this path and write sampled status back into it.

4.2 AXI4 flavour datapath

The AXI4 flavour replaces the streaming middle with a memory-to-memory job chain across three job memories.

4.2.1 Figure 2.2: BCH AXI4 memory-to-memory datapath



BCH AXI4 memory-to-memory datapath

Source: 02_bch_axi4_loop.dot

4.3 Clock and reset

Clock and reset come from `bch_loop_genesys2_top`: the 200 MHz LVDS system clock passes through `IBUFDS` and an `MMCME2_BASE` with `CLKFBOUT_MULT_F = 6` and `CLKOUT0_DIVIDE_F = 12`, giving a 100 MHz harness clock. The Nexys A7 top runs the same 100 MHz directly. That frequency was chosen because the BCH loop and the UART divisor both close timing comfortably at 100 MHz on the k325t-2, and it keeps the AXIS and AXI4 flavours interchangeable from the host's point of view.

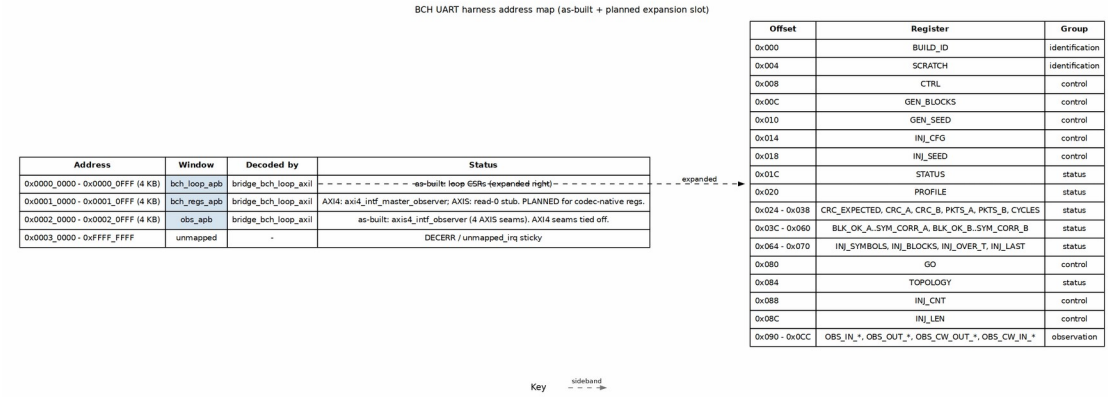
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Address Map

The harness exposes one 32-bit AXI4-Lite slave through the UART bridge. The generated `bridge_bch_loop_axil` fabric decodes three APB windows; the first window holds the loop's own CSR block. The host code in `build-loop/host/bch_loop.py` knows only two raw addresses — `OBS_AXI4_BASE = 0x00010000` and `OBS_AXIS_BASE = 0x00020000` — everything else is by-name access through the PeakRDL-generated `bch_loop_regs_regmap.py`.

5.1 Figure 2.4: UART harness address map



BCH UART harness address map

Source: 04_address_map.dot

5.2 Window summary

Base	Range	Size	Window	As-built use
0x0000_0000	0x0000_0000 - 0x0000_0FFF	4 KB	`bch_loop_apb`	Loop CSRs (`bch_loop_regs`) — expanded below.
0x0001_0000	0x0001_0000 - 0x0001_0FFF	4 KB	`bch_regs_apb`	**PLANNED:** codec-native register block. **As-built:** AXI4 flavour hosts `axi4_intf_master_observer`; AXIS flavour returns read-0.
0x0002_0000	0x0002_0000 - 0x0002_0FFF	4 KB	`obs_apb`	`axis4_intf_observer` on the four AXIS seams. On the AXI4 flavour the seams are tied off and reads return 0.
0x0003_0000+	—	—	unmapped	Sticky `unmapped_irq` in the bridge.

5.3 Loop CSR registers (window 0)

All registers are 32 bits and word-aligned. The address space is defined in `build-loop/rtl/bch_loop_regs.rdl` and generated into `build-loop/rtl/generated/bch_loop_regs/rtl/bch_loop_regs.sv` plus `build-loop/dv/tbclasses/bch_loop_regs_regmap.py`.

5.3.1 Identification and control

Offset	Register	Access	Purpose
0x000	`BUILD_ID`	R	ASCII identifier: `0x4243_4850` “BCHP” (board) or `0x4243_4853` “BCHS” (small).
0x004	`SCRATCH`	RW	Host link sanity.
0x008	`CTRL`	W / RW	`soft_reset`, `clear`, `bypass`, `throttle_a/b`, `inj_seed_on_start`.
0x080	`GO`	W	Pulse `start` to run `GEN_BLOCKS` blocks.
0x00C	`GEN_BLOCKS`	RW	Blocks per run.
0x010	`GEN_SEED`	RW	Pattern generator LFSR seed.
0x014	`INJ_CFG`	RW	Injector mode (`0..7`), error count, rate.
0x018	`INJ_SEED`	RW	Injector LFSR seed.
0x088	`INJ_CNT`	RW	Cluster mode min/max cluster count.
0x08C	`INJ_LEN`	RW	Cluster mode min/max cluster length.

5.3.2 Status and profile

Offset	Register	Access	Purpose
0x084	TOPOLOGY	R	decoders, kes_a, iface (AXIS vs AXI4).
0x020	PROFILE	R	n, t, m, spb (bytes per beat).
0x01C	STATUS	R	busy, gen_done, chk_a_done, data_err_a, crc_a_ok, AXI4 stage/overflow.
0x024 - 0x038	CRC_EXPECTED, CRC_A, CRC_B, PKTS_A, PKTS_B, CYCLES	R	Run-level CRC, packet and cycle counts.
0x03C - 0x060	BLK_OK_A..SYM_COR R_A, BLK_OK_B..SYM_COR R_B	R	Decoder verdict tallies; B-side tied off.
0x064 - 0x070	INJ_SYMBOLS, INJ_BLOCKS, INJ_OVER_T, INJ_LAST	R	Injector statistics.

5.3.3 Bandwidth meters

Offset	Register	Purpose
0x090 - 0x09C	OBS_IN_*	Codec input end: productive / backpressure / starvation / idle cycles.
0x0A0 - 0x0AC	OBS_OUT_*	Codec output end meters.
0x0B0 - 0x0BC	OBS_CW_OUT_*	Codeword out of encoder meters.
0x0C0 - 0x0CC	OBS_CW_IN_*	Codeword into decoder

Offset	Register	Purpose
		meters.

5.4 Observer windows (windows 1 and 2)

The observer blocks run their own copy of the shared `obs_regs` register block (`projects/components/utility-ip/misc/rtl/regs/generated/obs_regs_top_regmap.py`). The host attaches a second `UartRegisterMap` instance at each window base to read observer statistics by name. See `BchLoopDriver.axis_observer()` and `BchLoopDriver.axi4_observer()` in `build-loop/host/bch_loop.py`.

6 The FPGA-Testing Choices, And Why

6.1 Deterministic campaigns

Every board campaign is deterministic: the default `gen_seed` and `inj_seed` are 1, and the sequences in `bin/seq_*.py` produce the same bit-exact runs in cocotb simulation and on the board. That matters because a failure you can replay is a failure you can fix; a non-deterministic board-only failure is just a scary story. `seq_random.py` still varies its draws, but those draws come from a host-side `random.Random(seed)` seeded by the caller, so the variation is reproducible when you need it.

6.2 Injector modes mapped to real failure mechanisms

The shared `error_injector` exposes eight modes, and each one answers a different physical question. Exact-count mode proves the correction boundary: at `e <= t` every block must correct with exactly `e` bits flipped, and at `e > t` the decoder must flag uncorrectable. Burst and rate modes model raw bit-error behaviour on a noisy channel. Clusters and localized mode stand in for spatially correlated faults — a bad DRAM row, a disturbed NAND page, a half-plane failure — because real errors are rarely uniform. Badblock mode models retention tails and worn blocks: most of the stream stays clean while a hard minority runs at a much higher error rate. Debug mode is the bring-up helper, a deterministic walking pattern that closes by inspection.

6.3 Post-encoder injection protecting parity bits too

The injector sits after the encoder, not inside the generator. That is not an accident: an error in the generator data would be encoded faithfully and the code would never see it. By corrupting the codeword, the parity bits are exposed to the same errors as the data, which is what happens when a real packet is stored, transmitted, or read back from memory. The `bch_loop_harness.sv` header comment calls this out explicitly, and the layout follows from it.

6.4 Recheck + flip-only verdict as the on-chip oracle

The BCH decoder uses riBM with odd-syndrome computation and even syndromes by squaring. It runs a second-syndrome recheck (`ENABLE_RECHECK`) and performs flip-only correction — there is no Forney, because binary values are always 1. The checker independently regenerates the expected stream and reports `data_err` on any mismatched beat. Together they form the on-chip oracle: the decoder decides whether it thinks it corrected the block, and the checker decides whether the recovered bytes actually match the reference.

6.5 UART over a debug core, plus the safety pair

The host link is a plain UART at 115200 baud, driven by `uart_axil_bridge`. That choice keeps the harness scriptable from Python without a debug core licence and without vendor-specific probes. But scriptable does not mean careless. Two pieces of shared tooling guard the board: `projects/fpga-systems/bin/board_lock.sh` serializes programming flows by JTAG serial, and `projects/fpga-systems/bin/fpga_board.py` refuses to program until a JTAG readback confirms the expected Genesys 2 serial `200300B818A0` on the chain with a device behind it. The readback also bounces `hw_server` and retries up to five times on the deviceless-enumeration race that otherwise refused roughly half of programming attempts when a second Digilent board shared the chain.

6.6 AXIS vs AXI4 flavours covering both integration styles

Two bitstreams are built and both are exercised. The AXIS flavour is a single streaming pipe; the AXI4 flavour is a memory-to-memory job chain. The AXI4 flavour refuses runs whose `GEN_BLOCKS` exceed the job-memory capacity rather than letting regions wrap, so long soaks belong on the AXIS image. The host reads `TOPOLOGY` to discover which datapath is loaded, so the same campaign scripts drive both without a command-line flag.

7 Operating It

Start with `make help` in `projects/fpga-systems/Genesys2/ecc-ip/bch/` and the per-build help under `build-loop/`. A typical board run looks like:

```
bin/run_smoke.py --board genesys2 --port /dev/ttyUSB0 --sequences init
smoke sweep
```

For the Nexys A7 small profile, build the bitstream with `make -C build-loop bitstream BCH_TARGET=nexys_a7_100t BCH_PROFILE=small` and drive the same sequences over the A7's UART; `init` proves the "BCHS" BUILD_ID and the PROFILE CSR reports n=248, t=3, so the host programs pick up the geometry from the board.

Board timing, utilisation, and the matrix summary are recorded in `stable/MANIFEST.md` and `stable/reports/{genesys2_axis,genesys2_axi4}/`. Campaign transcripts turn into reports with `projects/fpga-systems/bin/report_battery.py` (JSON artifacts, correction-boundary / decode-cost / soak-timeline figures, and a generated `FINDINGS.md`); published runs live under `stable/results/`. The run evidence — the 2026-10-05 battery and the one-million-block soak — is rolled up in the companion *Binary BCH Board Validation Report* (`docs/Binary_BCH_Board_Validation_v0.1.pdf`). For the Galois-field math behind the profile, see the component HAS chapter 7 under `projects/components/ecc-ip/bch/docs/`. The sister harness has the matching guide at `../../reed-solomon/docs/Reed_Solomon_UART_Harness_v0.1.pdf`.

8 What Is Deliberately Not Board-Tested

Some things are out of scope, and it is worth saying so plainly.

- BCH has no erasure path in this harness: the injector's `out_erasure` is tied off and `cfg_mark_erasure` is held low.
- The board validates only the `BCH(4224,4120) t=8` profile; smaller profiles like `(63,57) t=1` are covered in simulation and component DV, not on the board.
- The component DV matrix exercises the codec in ways the board harness does not replicate.
- The 2026-10-05 million-block soak is statistical, not exhaustive; its counters and the bounded mis-decode criterion are reported in the Board Validation Report.